Computing Systems and Programming (2): iocontrol_serialout

Dr. Clifford Choy

School of Design

16 Oct 2026

iocontrol_serialout

A minimal program

Output | Serial Monitor ×

Message (Enter to send message to 'AirM2M_CORE_ESP32C3' on 'CO

```
chip started
current time=10
current time=100
current time=200
current time=300
current time=400
current time=500
current time=600
current time=700
current time=800
current time=900
current time=1000
current time=1100
current time=1200
current time=1300
current time=1400
current time=1500
current time=1600
current time=1700
current time=1800
current time=1900
current time=2000
current time=2100
current time=2200
current time=2300
current time=2400
```

iocontrol_serialout.ino

```
//
//  for demonstration use of handler function in IOControl for simple printing on Serial Monitor
//  serve as a minimal example for the library
//    board:
//      Luatos ESP32C3 board
//      ESP32S3 Dev Module
//    circuit:
//      none
//

#include <IO_Control.h>

DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for scheduler
  //  hdlr - name of the handler function
  //  _ctrl - Control object
  //  IOControl - class of object of this handler
  //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoked

  // if (_ctrl.isSchedulerTimeEvent())
  //     Serial.println("IOControl - SchedulerTimerEvent");
  Serial.print("current time=");
  Serial.println(_ctrl.getUpdateTime());
});

void setup()
{
  // put your setup code here, to run once:
  IO_Control.begin();

  Serial.begin(115200);
  Serial.println("chip started");

  IO_Control.enableScheduler(0, 100, hdlr);
    //  0 - start time of hdlr
    //  100 - interval between consecutive invocation of hdlr
    //  hdlr - must be declared using DeclareHandler(...)
}

void loop() {
  // put your main code here, to run repeatedly:
}
```

iocontrol_serialout

- Comments // or /* ... */
- Ignored by compiler
- grey in color
- Use **Edit > Comment/uncomment** to toggle

- Meaning: use the **IO Control** library
- Literally “include” or insert source code from the file **IO_Control.h** into this program
- Name of the library is **IO_Control**, with space replaced by underscore, replace space with underscore, as the compiler will interpret **IO Control** as two separate things due to presence of a space

iocontrol_serialout.ino

```
1  //
2  //  for demonstration use of handler function in IOControl for simple printing on Serial Monitor
3  //  serve as a minimal example for the library
4  //    board:
5  //      Luatos ESP32C3 board
6  //      ESP32S3 Dev Module
7  //    circuit:
8  //      none
9  //
10
11 #include <IO_Control.h>
12
13 DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for scheduler
14   //  hdlr - name of the handler function
15   //  _ctrl - Control object
16   //  IOControl - class of object of this handler
17   //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoked
18
19   // if (_ctrl.isSchedulerTimeEvent())
20   //     Serial.println("IOControl - SchedulerTimerEvent");
21   Serial.print("current time=");
22   Serial.println(_ctrl.getUpdateTime());
23 });
24
25 void setup()
26 {
27   // put your setup code here, to run once:
28   IO_Control.begin();
29
30   Serial.begin(115200);
31   Serial.println("chip started");
32
33   IO_Control.enableScheduler(0, 100, hdlr);
34     //  0 - start time of hdlr
35     //  100 - interval between consecutive invocation of hdlr
36     //  hdlr - must be declared using DeclareHandler(...)
37 }
38
39 void loop() {
40   // put your main code here, to run repeatedly:
41 }
```

iocontrol_serialout

- One **statement**
- Almost always end with a semi-colon **;**
- Each statement instructs ESP32C3 to perform specific action

- **Block** of statements (codes)
- Enclosed by curly braces
- `{` for opening and `}` for closing

- Define a function `void setup()`, with the block of statements

- Define a function `void loop()`, with the block of statements
- **Recommend NOT to add anything here, as will affect overall performance of other parts of the program**

```
11  #include <IO_Control.h>
12
13  DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for scheduler
14    //  hdlr - name of the handler function
15    //  _ctrl - Control object
16    //  IOControl - class of object of this handler
17    //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoked
18
19    // if (_ctrl.isSchedulerTimeEvent())
20    //     Serial.println("IOControl - SchedulerTimerEvent");
21    Serial.print("current time=");
22    Serial.println(_ctrl.getUpdateTime());
23  });
24
25  void setup()
26  {
27    // put your setup code here, to run once:
28    IO_Control.begin();
29
30    Serial.begin(115200);
31    Serial.println("chip started");
32
33    IO_Control.enableScheduler(0, 100, hdlr);
34      //  0 - start time of hdlr
35      //  100 - interval between consecutive invocation of hdlr
36      //  hdlr - must be declared using DeclareHandler(...)
37  }
38
39  void loop() {
40    // put your main code here, to run repeatedly:
41  }
```

iocontrol_serialout

- Execute or **invoke** once only after ESP32C3 reset

- Setup/**initialize** the IO Control library
- `IO_Control` is an **object** already defined by default, which is used to access some basic functions of IO Control library
- Only invoke once

- `Serial` is an object already defined by default, which corresponds to Serial interface
- Setup/**initialize** serial connection with **Serial Monitor** on PC, and print something

- Start a **scheduler** to invoke a **handler function** repetitively
- Name of the **handler function** is `hdlr`
- Starting at time = 0 (means “immediate” or “as soon as possible”)
- Invoke handler function every 100ms (0.1s)

```
11  #include <IO_Control.h>
12
13  DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for scheduler
14    //  hdlr - name of the handler function
15    //  _ctrl - Control object
16    //  IOControl - class of object of this handler
17    //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoked
18
19    // if (_ctrl.isSchedulerTimeEvent())
20    //     Serial.println("IOControl - SchedulerTimerEvent");
21    Serial.print("current time=");
22    Serial.println(_ctrl.getUpdateTime());
23  });
24
25  void setup()
26  {
27    // put your setup code here, to run once:
28    IO_Control.begin();
29
30    Serial.begin(115200);
31    Serial.println("chip started");
32
33    IO_Control.enableScheduler(0, 100, hdlr);
34      //  0 - start time of hdlr
35      //  100 - interval between consecutive invocation of hdlr
36      //  hdlr - must be declared using DeclareHandler(...)
37  }
38
39  void loop() {
40    // put your main code here, to run repeatedly:
41  }
```

Serial Communication, Serial Monitor, Serial

```
void setup()
{
  // put your setup code here, to run once:
  IO_Control.begin();

  Serial.begin(115200);
  Serial.println("chip started");

  IO_Control.enableScheduler(0, 100, hdlr);
}
```

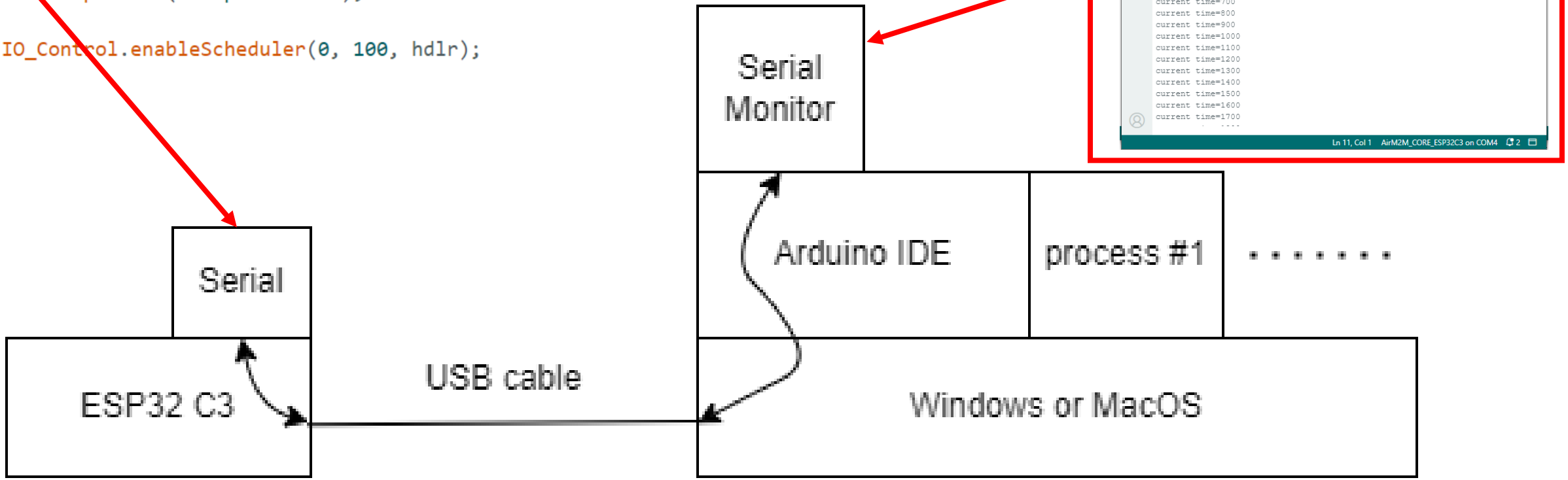
```
4 {
5   // if (_ctrl.isSchedulerTimeEvent(_status))
6   //     Serial..println("IOControl - SchedulerT
7   Serial.print("current time=");
8   Serial.println(_currentMillis);
9 }
10 DeclareHandlerEnd(hdlr,_ctrl,IOControl,_currentM
```

Serial Monitor × Output

Message (Enter to send message to 'AirM2M_CORE_ES... | No Line Ending | 115200 baud

```
chip started
current time=87
current time=100
current time=200
current time=300
current time=400
current time=500
current time=600
current time=700
current time=800
current time=900
current time=1000
current time=1100
current time=1200
current time=1300
current time=1400
current time=1500
current time=1600
current time=1700
```

Ln 11, Col 1 AirM2M_CORE_ESP32C3 on COM4



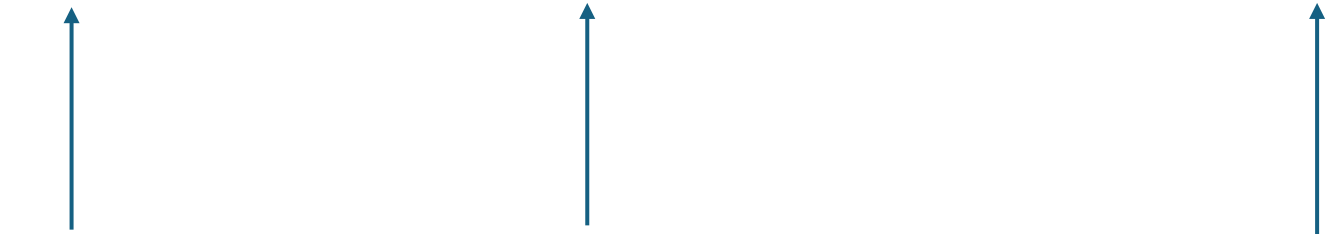
`Serial.println("current time=");`

- `Serial` – an object defined by default, corresponding to the Serial Interface on the ESP32C3 (for communicating with the PC through its USB interface)
- `println("current time=");`
  - A **function** or a **method** of `Serial`
  - Refer to https://docs.arduino.cc/language-reference/en/functions/communication/serial/println/

- Meaning:
  - invoke a function/method (`print(…)`) of the object `Serial`

Object, Function, Arguments

```
Serial.println("current time=");
```



object

function/method

argument
Possible to have no argument
If more than one, separated by ","
Number of arguments depend on specification of the function

```
IOControl.enableScheduler(0,100,hdlr);
```

- https://www.designandmake.org/display/DeFL/Class+Control#ClassControl-enableScheduler(...)

- Invoke the handler function `hdlr`, every 100ms, immediately after start
- `hdlr` has to be declared using `DefineHandler`

iocontrol_serialout

- **Handler function** is defined here
- Note: enclosed in **braces { }**, and inside **brackets ( )**

- For declaring the **handler function**
- `DefineHandler` has 4 **arguments**, enclosed within a pair of brackets
  - 1st : name of the handler function, which is `hdlr`
  - 2nd : name of the object whose handler function is invoked (`_ctrl`)
  - 3rd : name of the **class** of the object (`IOControl`)
  - 4th: **handler function** which is invoked by the scheduler, within pair of braces `{ }`

- Name of the function to be invoked by the **scheduler** is `hdlr` - should be the same as name of the handler function defined above via `DeclareHandler(...)`

```
11  #include <IO_Control.h>
12
13  DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for schedule
14    //  hdlr - name of the handler function
15    //  _ctrl - Control object
16    //  IOControl - class of object of this handler
17    //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is invoke
18
19    // if (_ctrl.isSchedulerTimeEvent())
20    //     Serial.println("IOControl - SchedulerTimerEvent");
21    Serial.print("current time=");
22    Serial.println(_ctrl.getUpdateTime());
23  });
24
25  void setup()
26  {
27    // put your setup code here, to run once:
28    IO_Control.begin();
29
30    Serial.begin(115200);
31    Serial.println("chip started");
32
33    IO_Control.enableScheduler(0, 100, hdlr)
34      //  0 - start time of hdlr
35      //  100 - interval between consecutive invocation of hdlr
36      //  hdlr - must be declared using DeclareHandler(...)
37  }
38
39  void loop() {
40    // put your main code here, to run repeatedly:
41  }
```

Basic Concepts (1)

- A program consists of at least one statements
- A program is called “sketch” in Arduino, and with “.ino” extension
- Always under a folder of the same name but without extension
  - E.g. “iocontrol_serialout.ino” is under a folder “iocontrol_serialout”

- Almost all statements end with a semi-comma
- Statements can be grouped into a block using braces {}

Basic Concepts (2)

- `void setup()`
  - A function with one or more statements
  - Execute once after reset
  - Execute statement one by one, according to order in the program

- `void loop()`
  - A function with one or more statements
  - Execute repeatedly after `setup()`, and repeat forever until reset
  - When using the IO Control library, most likely do not have to add other codes in here, as most of them should be in different “handler functions”

Basic Concepts (3) - Scheduler and Handler

- A CPU can only perform one thing at a time, although it is running very fast
  - Example:
    - ESP32C3 running at 160MHz, it can execute 160 million **machine instructions** per second
    - Executing one statement requires more than one machine instruction
    - Hence, 160MHz **does not** mean 160 million **statements** per second
- Scheduler
  - Responsible for determining **which tasks** to be executed by **when**
- Handler or handler function
  - A function with statements for performing something useful
  - A handler function must be defined using `DeclareHandler`
  - Use `enableScheduler` with the **same function name** as declared using `DeclareHandler` to let the scheduler knows which function to **invoke** or **call**
  - Scheduler allows the handler function to execute when the time for the task to be executed has come

Basic Concepts (4) - Scheduler and Handler

- `IO_Control.enableScheduler(0,100,hdlr);`
  - Associate `hdlr` with `IO_Control`
  - Only one handler function can be associated with an object
- Every 100ms, the code defined in `DeclareHandler` will be executed with `_ctrl` = `IO_Control`

- NOTE:
  - `IO_Control` – an object
  - `IOControl` – a class

```
//
//  for demonstration use of handler function in IOControl for simpl
//  serve as a minimal example for the library
//    board:
//      Luatos ESP32C3 board
//      ESP32S3 Dev Module
//    circuit:
//      none
//

#include <IO_Control.h>

DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for sche
  //  hdlr - name of the handler function
  //  _ctrl - Control object
  //  IOControl - class of object of this handler
  //  _ctrl.getUpdateTime() (ulong_t) - time when this handler is in

  // if (_ctrl.isSchedulerTimeEvent())
  //     Serial.println("IOControl - SchedulerTimerEvent");
  Serial.print("current time=");
  Serial.println(_ctrl.getUpdateTime());
});

void setup()
{
  // put your setup code here, to run once:
  IO_Control.begin();

  Serial.begin(115200);
  Serial.println("chip started");

  IO_Control.enableScheduler(0, 100, hdlr);
    //  0 - start time of hdlr
    //  100 - interval between consecutive invocation of hdlr
    //  hdlr - must be declared using DeclareHandler(...)
}

void loop() {
  // put your main code here, to run repeatedly:
}
```

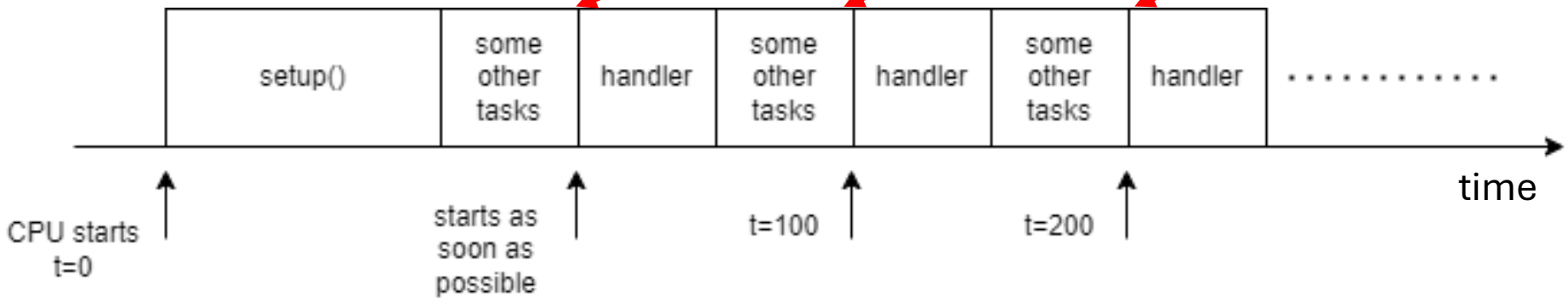
Basic Concepts (4) - Scheduler and Handler

```
13  DeclareHandler(hdlr,_ctrl,IOControl,{ //  declare a handler for scheduler
14    //  hdlr - name of the hand
15    //  _ctrl - Control object
16    //  IOControl - class of ob
17                                  me() (ulong_t) - time when this handler is invoked
18
19    // if (_ctrl.isSchedulerTimeEvent())
20    //     Serial.println("IOControl - SchedulerTimerEvent");
21    Serial.print("current time=");
22    Serial.println(_ctrl.getUpdateTime());
23  });
```

a **variable** of `_ctrl` (or `IOControl`) with value corresponding to time (in milliseconds) when handler is invoked

`_ctrl` is `IO_Control`

```
Output   Serial Monitor ×
Message (Enter to send message
chip started
current time=10
current time=100
current time=200
current time=300
current time=400
current time=500
current time=600
current time=700
current time=800
current time=900
current time=1000
current time=1100
current time=1200
current time=1300
current time=1400
current time=1500
current time=1600
current time=1700
```



Basic Concepts (5)

- A program consists of **keywords** and **names**
- Names must be declared before use, or else error in compilation will occur
- Some names are declared by default
  - `Serial`
  - `IO_Control`
- Some names must be declared in the program
  - `void setup()`
  - `void loop()`
  - `hdlr`

Basic Concepts (6) - Objects

- Think of them as “helpers” which can help you to perform different tasks
- Some names are used to refer to objects
  - `Serial`
  - `IO_Control`
- `Serial`
  - An object for handling serial communication
- `Serial.print( … )`
  - Invoke a `print` function in `Serial`, which sends out a message to the serial communication interface to the PC
- `Serial.println( … )`
  - Similar to `Serial.print( … )`, but send out a “new line” after sending the message

Task 1

- Modify the program “iocontrol_serialout” as follows:
  - Change name of the handler to “iocontrolHandler”
  - Change the starting time so that the handler will be invoked for the first time 1.5s after the CPU starts
  - Print the message “Welcome, current time since start = *<time in milliseconds>*” where *<time in milliseconds>* is the current time in milliseconds
- Compile and upload your program to ESP32C3

- Under the heading “Task 1” in your submission page, upload the following:
  - Screen-capture of the output from Serial Monitor
  - Compressed file of your sketch (in zip format)

Summary

- Comments
- `setup()`, `loop()`
- Statement, block, function/method
- `IO_Control`, `Serial`
- `DeclareHandler`